Figure 8.4 Each entity is represented by a `DbSet`, which is mapped to a table in the database.

8.1.6 Adding DbSets

First you will add a `DbSet` to the context for each entity you want to map to a database table. By convention, the table will take the name of the `DbSet` property. The next listing shows the three classes you have created in your model so far, each represented as a `DbSet<TEntity>`.

Listing 8.8 `DbSet` properties mapping to tables in a database

```
public class CityBreaksContext : DbContext
{
    public CityBreaksContext(DbContextOptions options) : base(options)
    {

    }
    public DbSet<City> Cities { get; set; }
    public DbSet<Country> Countries { get; set; }
    public DbSet<Property> Properties { get; set; }
}
```

8.1.7 Configuring the model

If you were to create a migration at this stage, it would generate a database with three tables, one for each `DbSet`, and it would use conventions to create columns based on the properties for each of the types represented by the `DbSet` type parameter. In most real-world applications, the default conventions will be acceptable for most of your model, especially if you are starting from scratch. For those cases in which the conventions aren't suitable or EF Core needs help understanding your intentions, EF Core provides configuration APIs that allow you to override the conventions.